

ECE 6775 Final Project Report

Mandelbrot Set Rendering Acceleration

Jay Chawrey (jpc369), Ashley Heckman (agh93), James Ong (jo445), Chuqiao Yao (cy528)

1 Introduction

Within the 21st century, hardware acceleration has become increasingly prevalent with domains ranging from computer vision to low latency trading. Reflecting this trend, this project leverages hardware acceleration for generating Mandelbrot set images, a task that proves to be computationally intensive due to the number of pixels and the high number of iterations required to calculate each pixel in the set. The Mandelbrot set is a renowned fractal defined by an iterative equation involving complex numbers. Throughout this project, FPGAs and the Vivado HLS toolchain were utilized to design a highly parallelized architecture that significantly enhances the efficiency of Mandelbrot set computations. By dividing the image into 30000 pixel locations with up to 1000 iterations per point, the power of hardware acceleration in handling resource-intensive calculations is demonstrated.

2 Problem Description

Each pixel in the Mandelbrot set is represented by a complex number c , which undergoes repeated iterations based on the equation shown in *Figure 1*. The iteration continues until the magnitude of Z_n exceeds a threshold or reaches the maximum number of iterations, which is set to 1000. Based on these results, pixels are classified as either convergent or divergent, with corresponding color codes assigned.

$$\begin{cases} z_0 = 0 \\ z_{n+1} = z_n^2 + c \end{cases}$$

Figure 1: Mandelbrot Set computing equation

The input to the system consists of the c values for each pixel, configurations for data types, and tile size parameters to explore trade-offs between resource utilization and computation speed. Outputs include a dataset indicating the classification of each pixel, validated against ground truth to ensure accuracy, and a Python script to visualize the generated Mandelbrot set.

The design leverages the inherent parallelism of Mandelbrot set calculations, as each pixel can be computed independently. Exploiting the symmetry of the Mandelbrot set along the real axis, only half the points need to be calculated, significantly reducing computational requirements. Using Vivado HLS and Allo, the FPGA architecture is implemented with pipelined and parallelized modules that efficiently handle these operations.

3 Methodology and Implementation

Three designs are synthesized in an attempt to effectively implement the Mandelbrot set computations. The baseline design serves as the starting point for all implementations. It enables us to investigate the effect of image and fixed point sizes on the quality of the rendered output image. It also enables us to obtain a ground truth output, which other implementations are compared against for accuracy. Building off the baseline, two different optimizations are implemented: tiling and Allo.

3.1 Baseline Design

The baseline design involves partitioning the subspace of the Mandelbrot set into a series of pixel locations, treating the x and y axes as real and imaginary parts of c , respectively. For each pixel, the number of iterations that it takes for the equation to diverge is calculated, then mapped to a color value. The iteration has diverged if the magnitude of Z_n is greater than 4. Initially, the loop order was row, column, count. However, it was obvious that this would not allow for parallelization, as each iteration of count depends on the previous iteration. Thus, the loop order was

changed to count, row, column. This loop ordering makes it easier to parallelize later, as the overall structure is already present. This did require extra arrays to keep track of the intermediate real and imaginary parts of Z_n for each pixel, but did not significantly harm performance.

Additionally, since the Mandelbrot set is symmetric about the real axis, only the top half of the Mandelbrot set needs to be generated [1]. The bottom half can be mirrored and displayed in a Python script. This significantly aided in reducing time and resource utilization for the Mandelbrot set, as only half of the pixels needed to be computed. This symmetry is demonstrated in *Figure 2*.

Next, the DUT was implemented. The instream signal is solely a synchronization signal that tells the DUT to start. The value of this stream does not matter; it only matters that there is data present. The outstream is more complex. Since the hls streams are capable of sending 32 bits per message and the color value for each pixel is 4 bits, 8 output values are packed into each hls outstream.

A flowchart showing the steps for the Mandelbrot computations is shown in *Figure 3*. A synchronization signal is sent to the DUT. The DUT then calls the Mandelbrot_compute function, which initializes necessary arrays then calculates the color value for each pixel. The generated value is checked against a known good run to ensure that image accuracy remains high.

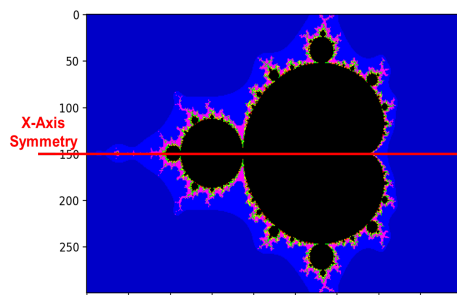


Figure 2: Mandelbrot Line of Symmetry

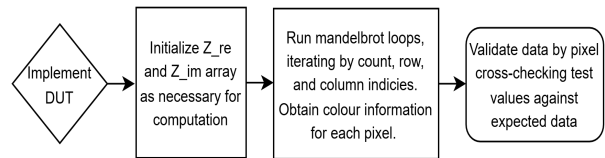


Figure 3: Program Flowchart

Once the baseline design was working as expected, fixed point types were explored. The smallest size that still produces a reliable image is $\langle 25, 5 \rangle$, shown in *Figure 4*. Many bigger sizes also produce this output, but this is the smallest (and therefore the best) size that produces this. Throughout the exploration, a couple of interesting effects were observed. *Figure 5* shows a design that has too little precision due to not enough fractional bits, and the line extending out to the left becomes less clearly defined. *Figure 6* has overflow issues due to only having 2 integer bits, and our divergence checking against a value of 4. Changing the data types did not impact performance or resource utilization at all, but it is still good to use the smallest size possible.

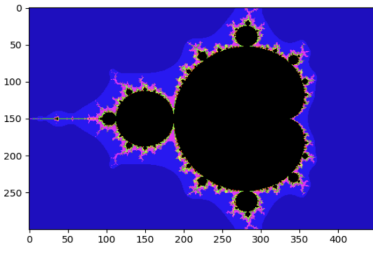


Figure 4: Expected Output

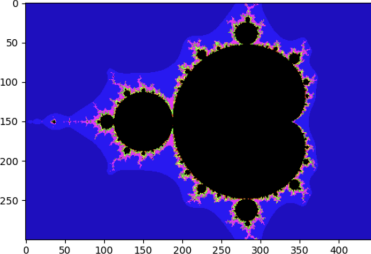


Figure 5: Not Enough Precision

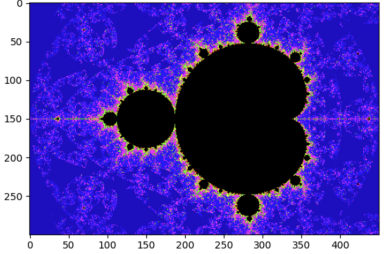


Figure 6: Overflow Issues

The final baseline design has a loop ordering of row, column, then count. It also has a fixed point size of $\langle 25, 5 \rangle$. This design allows for easy parallelization for later optimizations and uses a reasonable data type.

3.2 Optimized Design

The primary approach to optimization was inspired by Zhang which demonstrated the efficacy of tiling for accelerating FPGA convolution [2]. The current design uses intra-tile optimizations to partition the Mandelbrot subspace into a series of tiles which each contain a total of $(\text{tile_row} * \text{tile_col})$ pixels. Here, each pixel is computed in parallel through unrolling the nested loops, as demonstrated in Figure 7. For each column, there is a stride by a factor of tile_col to process the next tile, while incrementing the row index by a factor of tile_row to process the next series of row elements. Meanwhile the computation of each tile is pipelined along the column axis, resulting in an initiation interval of 1.

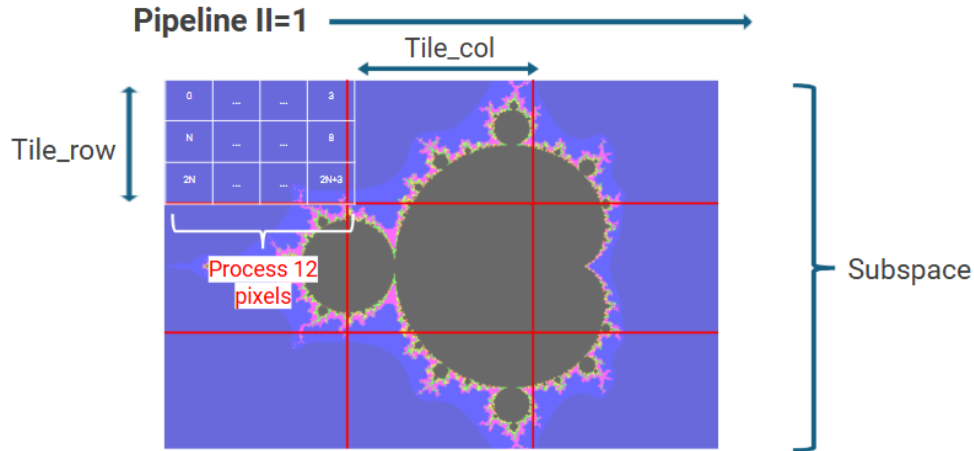


Figure 7: Conducting loop tiling which partitions the space into equal sized tiles of dimension $\text{tile_row} * \text{tile_col}$. The elements within each tile would be computed in parallel.

While tiling was the primary optimization explored, other optimizations were utilized for design synthesis. The baseline design leverages much conditional logic and break statements within the core computations. Attempting to pipeline the baseline results in large resource utilization, resulting in this design being scrapped due to the large multiplexor resource demand. Instead, the current Mandelbrot computation uses minimal conditional logic in the core compute, writing to an intermediate array count and following this using a function **remap_output()** aimed at mapping this intermediate array to a color code for Mandelbrot inspection.

3.3 System Architecture

The core architecture for this design was primarily inspired from the Cordic tutorial setup guide [3]. The system architecture for this implementation is relatively simplistic, and leverages an ARM32-CPU to implement a

host-program which communicates with an external Zynq FPGA using a series of input and output FIFO blocks. Meanwhile, it has a DUT block to communicate with the Mandelbrot compute function which calls tiling and remap output optimizations, respectively. *Figure 8* illustrates the system architecture which was used to synthesize our design.

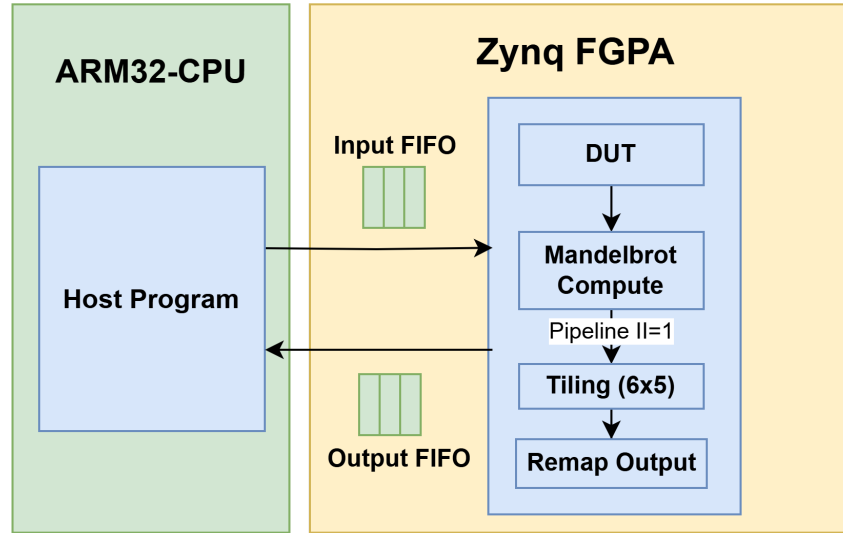


Figure 8: Relevant hardware blocks as necessary for running and synthesizing code on the Zynq based FPGA.

3.4 Allo Design

The Mandelbrot computation is implemented and optimized using the Allo framework, a novel programming model designed for composable accelerator design. Allo enables high-performance hardware generation through a Pythonic interface, providing an abstraction over traditional HLS workflows. The framework simplifies hardware design by decoupling hardware customizations (e.g., loop pipelining, unrolling, memory partitioning) from the algorithm specification, offering a productive and flexible environment for creating complex designs. In this project, Allo's capabilities are leveraged to design and optimize the Mandelbrot computation kernel for FPGA hardware, focusing on achieving a balance between performance, resource efficiency, and scalability. Below are the details of the design and optimization process, including the motivation behind each decision.

3.4.1 Understanding Allo and Its Advantages

Allo was chosen as the framework for this project due to its ability to:

1. **Provide Pythonic simplicity:** Unlike traditional HLS workflows requiring intricate vendor-specific pragmas, Allo allows the designer to focus on algorithm logic while expressing hardware-specific optimizations in a high-level, intuitive manner.
2. **Encapsulate customizations:** Through decoupled primitives for loop transformations, memory partitioning, and pipelining, Allo enables fine-grained control over hardware performance optimizations.
3. **Support composability:** The `.compose()` primitive allows the designer to modularize the Mandelbrot design into reusable building blocks, making it easy to test and optimize different components independently.
4. **Simplified Project Creation:** In Allo, only the kernel needs to be defined, as the framework automatically handles the generation of the entire project, including components like the host file, ZedBoard integration, and other essential parts, making the development process highly efficient.

3.4.2 Implementation Details

1. Framework and Libraries:

The Allo framework serves as the foundation for both the baseline and optimized designs. The backend utilizes Vitis HLS for synthesis and simulation, with fixed-point arithmetic (ap_fixed) for precise and resource-efficient calculations.

2. Software Components: Allo's Python-based customization API, combined with numpy for testing.

3. Design Workflow:

Core PE (Processing Element): This unit handles the iterative Mandelbrot computation for a single coordinate, encapsulating complex arithmetic operations.

Compute Kernel: The Mandelbrot_compute function distributes the workload across multiple PEs, managing dataflow, and ensuring parallel computation across the grid.

4. Testing for Correctness with Pytest:

To ensure the correctness of the implementation, Pytest is used as a testing framework. The output of the Mandelbrot computation kernel is compared against precomputed expected results stored in a data file. Pytest facilitates automated testing by verifying that the generated results matched the expected values within a defined tolerance range, ensuring precision and correctness across all points in the computation grid. This approach allows for rapid regression testing, particularly after making optimizations or design changes.

3.4.3 Optimization Techniques and Design Rationale

To achieve the desired speedup, the design is iteratively analyzed and optimized using the following methods:

- **Loop Unrolling:** Unrolling the inner computation loop by a factor of 100 enables parallel execution across multiple grid columns, significantly reducing computation latency per row. The unrolling is implemented using Allo's s.unroll() primitive, which allows precise control over the level of parallelism.
- **Loop Pipelining:** Pipelining the iteration loop in the PE ensures overlapping of computations across cycles, reducing initiation intervals (II) and increasing throughput. Allo's s.pipeline() primitive is applied with an II of 2, balancing performance gains with resource constraints.
- **Array Partitioning:** Partitioning the output buffer along its second dimension facilitates concurrent memory writes, alleviating bottlenecks in memory access. This is achieved through the s.partition() primitive, creating multiple memory banks for parallel access.
- **Fixed-Point Arithmetic:** Switching from floating-point to fixed-point arithmetic (ap_fixed) reduces FPGA resource usage while maintaining sufficient precision for the iterative computations. Fixed-point types are specified for all real and imaginary calculations in the PE, carefully tuning their bit widths to match the algorithm's requirements.
- **PE Modularity and Composition:** Modularizing the PE logic allows for independent testing and optimization before integrating it into the top-level kernel. The .compose() primitive is used to integrate the PE with the kernel, ensuring a seamless connection while preserving modularity.
- **Iterative Refinement:** The design underwent multiple iterations, guided by profiling and resource utilization reports from Vitis HLS. For example, early simulations identified memory bottlenecks, leading to the introduction of array partitioning. Similarly, unrolling factors were fine-tuned to optimize resource utilization without exceeding FPGA constraints.

3.4.4 Dataflow and Design Architecture

The overall design is structured as follows:

- **Dataflow:** Input data (coordinate grid) is passed to an array of PEs. Each PE independently computes the Mandelbrot iterations for a given coordinate. Results are written to partitioned buffers and subsequently stored in external memory.
- **Interconnection Between Blocks:** PEs are organized in a pipelined structure with intermediate FIFOs for buffering. This ensures efficient data movement between computation and storage stages.

- **Parallelism:** Spatial Parallelism: Achieved through loop unrolling, allowing simultaneous computation across multiple columns.
- **Temporal Parallelism:** Realized through pipelining, enabling overlapping execution within each PE.

4 Evaluation

The DUT function has one input signal and output signal. The input signal simply takes any value as an input signal; this is used only for synchronization. The output signal passes all of the pixel colors between the host and the FPGA. There are several fixed constraints to keep the testing space consistent. The width is set to 300 pixels, the height is set to 200 pixels, the maximum number of iterations is fixed to 1000, and the range of explored c values are all fixed.

The baseline design was evaluated by running the Mandelbrot code then plotting the output. Once a good image was created, the color value for each pixel was saved in a .dat file. All future implementations were compared against that expected.dat file to check for accuracy and correctness. All designs achieved an accuracy of 100%. The FPGA design was evaluated by generating ten renderings at a time. Final reported execution times are the average value of ten different runs, with each run doing ten computations of the Mandelbrot set.

4.1 Synthesized Results

The simulated designs are quite promising. The execution times, shown in *Table 1*, show a speedup of 99x for the optimized design and 122x for the Allo design. Such a large speedup makes sense, as the baseline design doesn't take advantage of any parallelism; the pixels are computed one after another. With both the optimized and Allo designs, pixel-level parallelism is taken advantage of to compute multiple pixels at the same time.

Design	Maximum Latency (Cycles)	Average Latency (Cycles)	Speedup
Baseline (DUT)	180502606	113429428	—
Optimized (DUT)	1151427	1151427	99
Allo (DUT)	932409	932409	122
Baseline (Compute)	180412404	—	—
Optimized (Compute)	1061225	—	170

Table 1: Synthesized Execution Times

However, the speedup for the optimized design does not come for free. As seen in *Figure 9*, the optimized design uses more resources than the baseline design across the board. However, all resource utilizations are well under 85%, so there are no concerns about the design failing due to resource constraints.

Remarkably, the Allo design uses fewer resources than the baseline design in every category except for LUT usage. This is due to several reasons. Firstly, Allo restructures the code in a way that prevents most of the intra-loop dependencies, making dataflow far simpler. Secondly, the Allo design uses a row, column, count loop order, which removes the need for certain large arrays. Lastly, the HLS code has more intricate memory accesses than the Allo code does.

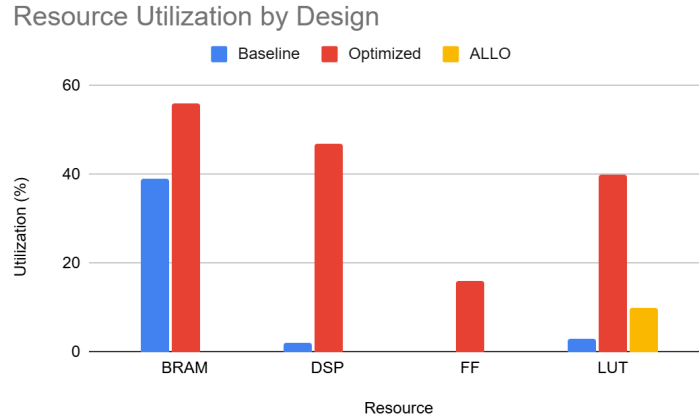


Figure 9: Resource Utilization by Design

4.2 FPGA Results

The overall FPGA execution time was disappointing. While the synthesized speedup was 99x for the optimized design, the FPGA was only able to obtain a 27x speedup. It was eventually discovered that this was due to communication overhead between the DUT and the host. In order to obtain a measure of the actual speedup achieved for the Mandelbrot computation, the DUT design was run without calling Mandelbrot_compute. This time was subtracted from the total execution time, yielding the execution time for just the Mandelbrot computation. The overall achieved speedup for the Mandelbrot computation was roughly 1600x, which is quite significant. This is also much larger than the synthesized speedup.

Design	SW Execution Time (msec)	Software Speedup	FPGA Execution Time: 10 runs (msec)	FPGA Speedup
Baseline (DUT)	231657.45	—	11340.94	—
Optimized (DUT)	612966.13	0.378	417.32	27
DUT Only	—	—	410.47	—
Baseline (Compute)	—	—	10923.61	—
Optimized (Compute)	—	—	6.85	1594

Table 2: Zedboard Execution Times

While the FPGA execution time got much better, the software execution time got worse. In fact, the software had a 2.65x slowdown. This makes sense, as a lot of complexity was added to the program throughout the optimizations. As a result, the code runs much faster in parallel, but when it's forced to run sequentially, it is slower.

5 Project Management

The project was completed through a collaborative effort, with each team member contributing to distinct tasks in line with the project timeline. Below is a detailed breakdown of the division of work, tasks, milestones, and the overall progression of the project.

In the initial phase, the group focused on individually exploring the Mandelbrot algorithm and how to synthesize this computation in Vivado. Notably, in this phase, Ashley developed the baseline synthesis which included running the Mandelbrot algorithm and creating the HLS and Zedboard environments. As a group, we also explored avenues for potential optimizations, which involved labeling the loops and finding bottleneck latencies. Notably, James explored optimization techniques such as fast-exit to the Mandelbrot set while Jay focused on exploring Allo synthesis.

During the second and third weeks, we explored optimization techniques. In this regard, Chuqiao and James focused on exploring different tiling methods to improve computational efficiency. James implemented a 2D matrix tiling design with code simplification that proved faster than Chuqiao's 1D array-based tiling method. Consequently, James's design was preferred and synthesized for execution in Vivado. Simultaneously, Ashley experimented with various variable types to evaluate their impact on speed, resource usage, and output fidelity. Meanwhile, Jay established a Python baseline and optimized model for Mandelbrot set computations on the Allo compiler. All tasks progressed as planned during this phase, and milestones were achieved on schedule.

In the final phase, the team explored the impact of different tile sizes on speed and resource usage. This experimentation encountered some challenges. A significant issue arose when generating the bitstream file for FPGA testing, which caused delays but was eventually resolved with Chuqiao and Ashley aiding in issue identification through progressive code rollbacks.

6 Conclusion

Within the scope of this project, the group synthesized a baseline design for Mandelbrot computation that iterated over each pixel sequentially, assigning the appropriate ground truth value. Several optimization strategies were explored, including tiling with code reordering and Allo synthesis. Ultimately, a 2D tiling approach with a tile size of 6x5 was selected for the final design, as it offered a balanced trade-off between resource utilization and hardware synthesizability for bitstream generation. Performance measurements revealed significant improvements over the baseline design. The tiled implementation achieved up to 170x maximal compute speedup and a 99x average speedup on the DUT, while Allo synthesis demonstrated a 122x average cycle speedup over the baseline, with simulation results outperforming the basic tiling implementation.

Furthermore, synthesizing the design on an FPGA demonstrated high performance. Here, the FPGA implementation achieved a 1594x speedup in compute time compared to the baseline. This underscores the effectiveness of the proposed optimizations and highlights the practical benefits of using HLS tools for accelerating and optimizing computationally intensive tasks. Altogether, the combination of 2D tiling and Allo synthesis proved highly effective for accelerating Mandelbrot set computations, achieving exceptional speedups across multiple benchmarks and demonstrating the value of high-level synthesis for hardware acceleration.

7. Acknowledgements

The success of this project was not solitary and we would like to attribute the success of the project to many of the course staff that provided useful contributions and suggestions all-throughout. First and foremost, we would like to thank Professor Zhiru Zhang for his supervision on this project, with his suggestions allowing us to understand the theory and implementation of various optimization approaches. Additionally, we would like to dedicate our thanks to Yixiao Du who provided useful feedback on Ed-discussion about optimization suggestions, and Hongzheng Chen who provided access to an Allo server which was invaluable for our synthesis and helped with setting up the synthesizer.

References:

1. Mahanta, Arun & Sarmah, Hemanta & Choudhury, Gautam. (2017). SOME STRUCTURAL AND DYNAMICAL PROPERTIES OF MANDELBROT SET. International Journal of Applied Mathematics and Statistical Science. 6. 35-58.
2. Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B. and Cong, J., 2015, February. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays (pp. 161-170).
3. Zhang, Z. Lab 3 Handout. Cornell University. Retrieved from <https://www.csl.cornell.edu/~zhiruz/5997/pdf/lab3-handout.pdf>
4. Chen, H., Zhang, N., Xiang, S., Zeng, Z., Dai, M., & Zhang, Z. (2024). Allo: A programming model for composable accelerator design. Proc. ACM Program. Lang., 8(PLDI, 171), Article 171. Association for Computing Machinery. <https://doi.org/10.1145/3656401>
5. [C100] H. Chen, N. Zhang, S. Xiang, Z. Zeng, M. Dai, and Z. Zhang, Allo: A Programming Model for Composable Accelerator Design, ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), Jun. 2024